

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: ITA 06

COURSE NAME: MACHINE LEARNING

CO4: K- Nearest Neighbour Learning – Locally weighted Regression – Radial Bases

Functions – Case Based Learning

Name: K. Dedeepya

Reg no: 192324252

1. Develop a Python program to implement Locally Weighted Regression. Given a dataset, compute weights based on distance and predict output for a query point. Use a simple kernel function and compare predictions with standard linear regression for better understanding.

Aim

To implement Locally Weighted Regression using a Gaussian kernel and compare its prediction with standard Linear Regression.

Theory

Locally Weighted Regression (LWR) is a non-parametric regression technique. It gives higher weights to data points closer to the query point and lower weights to distant points.

The Gaussian kernel is used to calculate the weights:

$$w = \exp(-d^2 / 2\tau^2)$$

where:

- d = distance between data point and query point
- τ = bandwidth parameter
- w = weight

Algorithm

1. Create a small dataset.
2. Select a query point.
3. Calculate the distance between each data point and the query.
4. Calculate weights using the Gaussian kernel.
5. Construct the weighted regression model.
6. Predict the output for the query point.
7. Train a standard Linear Regression model.

8. Compare both predictions.

Table – LWR vs Linear Regression

Feature	LWR	Linear Regression
Type	Local	Global
Weight	Distance-based	Equal
Kernel	Gaussian	Not used
Prediction	Based on nearby points	Based on complete dataset
Flexibility	High	Lower

Python Program

```
[1] ✓ 10
import numpy as np
from sklearn.linear_model import LinearRegression
X = np.array([1, 2, 3, 4, 5, 6]).reshape(-1, 1)
y = np.array([2, 4, 5, 8, 10, 12])
def gaussian_kernel(distance, tau):
    return np.exp(-(distance ** 2) / (2 * tau ** 2))
def locally_weighted_regression(X, y, query, tau=1.0):
    distances = X.flatten() - query
    weights = gaussian_kernel(distances, tau)
    W = np.diag(weights)
    X_design = np.c_[np.ones(len(X)), X]
    theta = np.linalg.pinv(X_design.T @ W @ X_design) @ \
        X_design.T @ W @ y
    query_design = np.array([1, query])
    prediction = query_design @ theta
    return prediction
query = 4.5
lwr_prediction = locally_weighted_regression(X, y, query)
model = LinearRegression()
model.fit(X, y)
linear_prediction = model.predict([[query]])[0]
print("Query Point:", query)
print("LWR Prediction:", round(lwr_prediction, 2))
print("Linear Regression Prediction:", round(linear_prediction, 2))

*** Query Point: 4.5
LWR Prediction: 8.88
Linear Regression Prediction: 8.86
```

Analysis

LWR focuses more on nearby data points, while Linear Regression considers the complete dataset equally. The value of τ controls how strongly nearby points influence the prediction.

Result

Locally Weighted Regression was successfully implemented using a Gaussian kernel and compared with standard Linear Regression.

2. Write a Python program to implement the Decision Tree algorithm for classification. Use a small dataset and construct the tree using information gain or Gini index. Allow prediction for test instances and display accuracy. Visualize the tree structure and analyse how feature selection impacts classification performance.

Aim

To implement a Decision Tree algorithm for classification using the Gini Index and evaluate its accuracy.

Theory

A Decision Tree is a supervised learning algorithm that classifies data by creating a tree-like structure. Each internal node represents a feature, each branch represents a decision, and each leaf represents a class.

The Gini Index is used to measure impurity.

$$\text{Gini} = 1 - \sum p_i^2$$

A lower Gini value indicates a purer node.

Algorithm

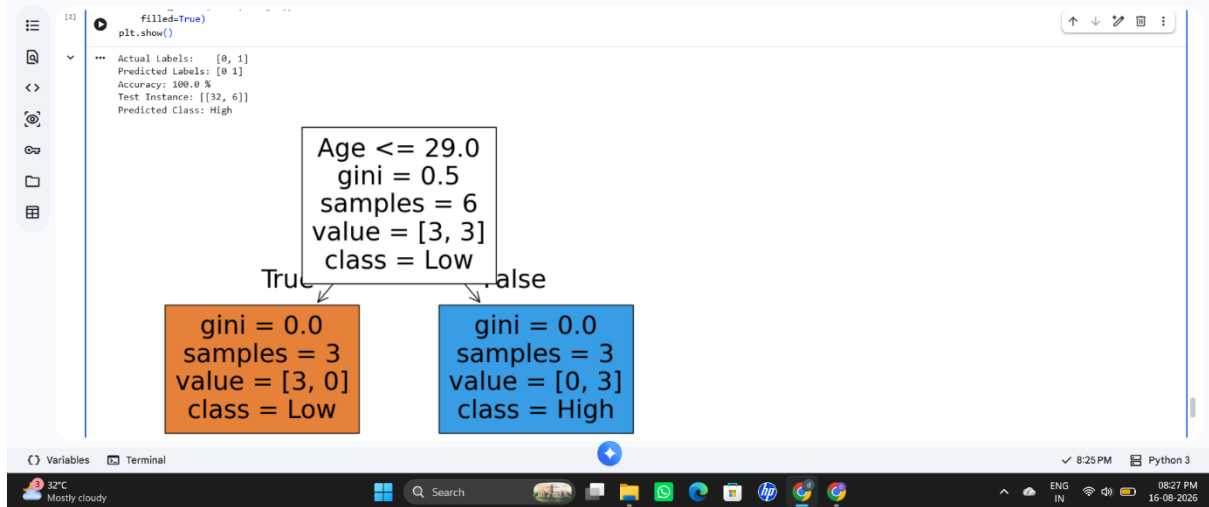
1. Create the dataset.
2. Split the dataset into training and testing data.
3. Create a Decision Tree classifier.
4. Use Gini Index as the splitting criterion.
5. Train the model.
6. Predict the test data.
7. Calculate accuracy.
8. Visualize the tree.

Flowchart



↓
End

Python Program



Analysis

The Decision Tree selects features that provide better class separation. The Gini Index helps select suitable splitting points. Important features improve classification, while excessive tree depth may cause overfitting.

Result

The Decision Tree classification algorithm was successfully implemented using the Gini Index, and the classification accuracy was obtained.

3. Create a Python program to implement the K-Means clustering algorithm. Initialize cluster centres, assign points to clusters, and update centroids iteratively. Display final clusters and visualize them using plots. Analyse how the number of clusters and initialization methods affect clustering results.

Aim

To implement the K-Means clustering algorithm, obtain clusters and centroids, and visualize the clustering results.

Theory

K-Means is an unsupervised machine learning algorithm. It divides data into a predefined number of clusters called K.

The algorithm repeatedly:

1. Selects initial centroids.
2. Assigns each point to its nearest centroid.
3. Calculates new centroids.

4. Repeats the process until the clusters become stable.

Algorithm

1. Create the dataset.
2. Select the number of clusters K.
3. Initialize cluster centroids.
4. Assign each point to the nearest centroid.
5. Calculate new centroids.
6. Repeat until convergence.
7. Display the final clusters.
8. Visualize the clusters.

Flowchart:

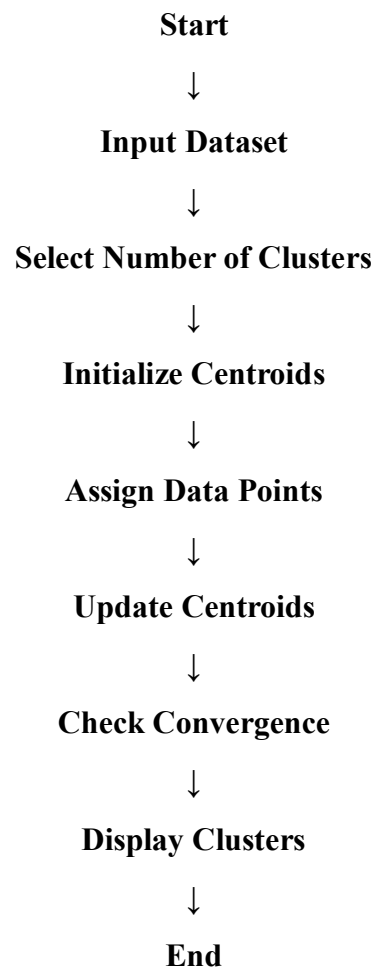
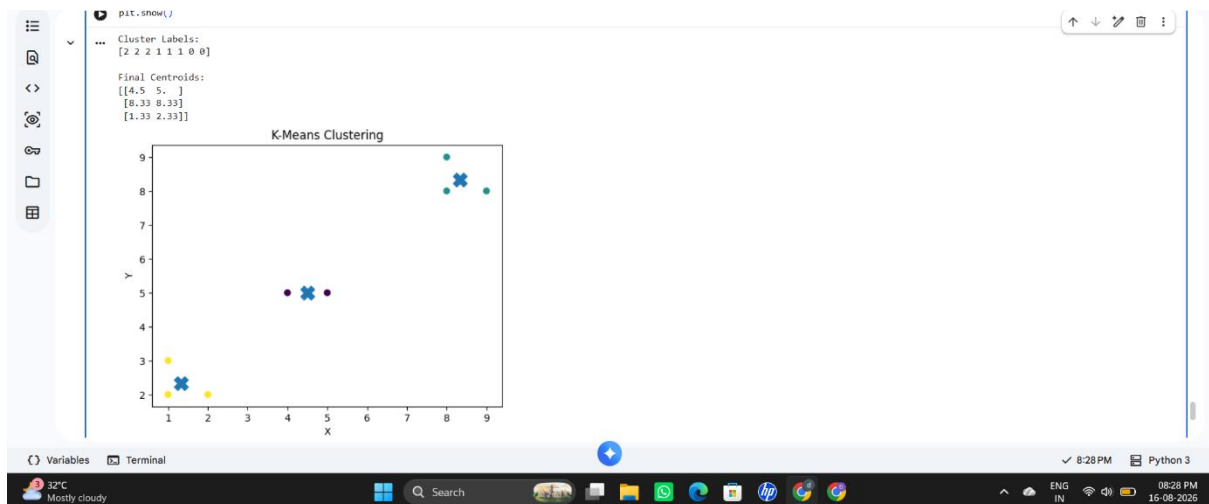


Table K- Means Steps

Step	Operation
1	Select number of clusters K

2	Initialize centroids
3	Assign points to nearest centroid
4	Calculate new centroids
5	Repeat until convergence

Python Program



Analysis

The value of K determines the number of clusters. Different values of K can produce different results. The initialization of centroids can also affect the final clustering.

Result

The K-Means clustering algorithm was successfully implemented and the final clusters and centroids were obtained and visualized.

4. Develop a Python program to implement Principal Component Analysis (PCA) for dimensionality reduction. Apply PCA on a dataset, reduce it to two principal components, and visualize the transformed data. Compare model performance before and after dimensionality reduction and analyse variance retention.

Aim

To implement Principal Component Analysis (PCA) for dimensionality reduction and reduce the dataset to two principal components.

Theory

Principal Component Analysis is a dimensionality reduction technique. It transforms the original features into new variables called **principal components**.

The first principal component contains the maximum variance, while the second contains the next highest variance.

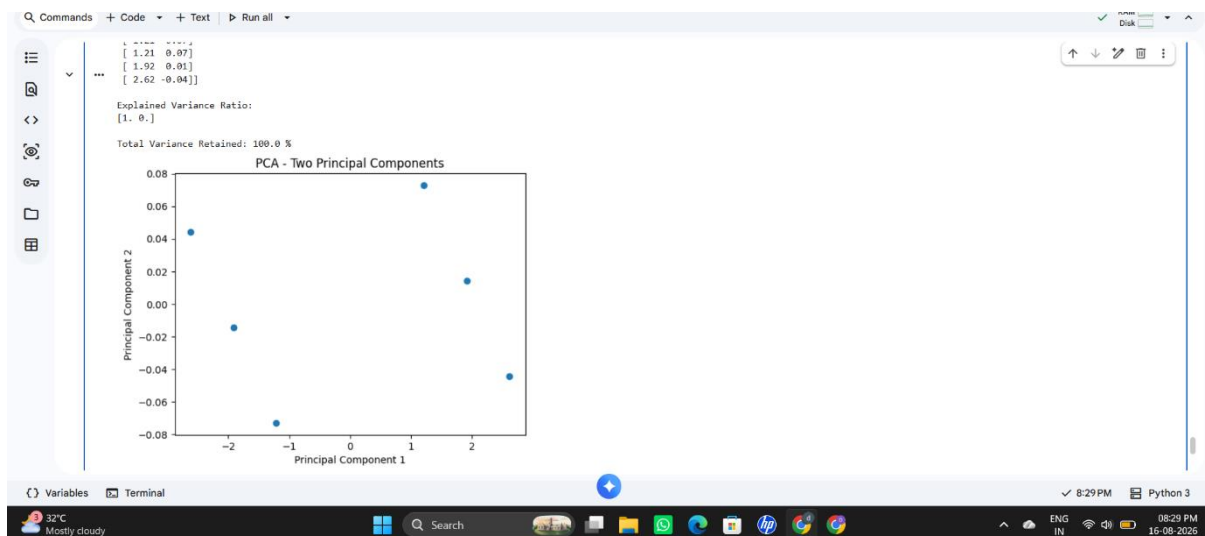
Algorithm

1. Create the dataset.
2. Standardize the data.
3. Apply PCA.
4. Select two principal components.
5. Transform the original data.
6. Calculate explained variance.
7. Visualize the transformed data.

Table PCA Components

Component	Description
PC1	Maximum variance
PC2	Second highest variance
Explained Variance	Information retained
Dimensionality Reduction	Reduces number of features

Python Program



Analysis

PCA reduces the number of features while retaining important information. Dimensionality reduction can make data easier to visualize and can reduce computational complexity.

Result

PCA was successfully implemented and the original four-dimensional dataset was reduced to two principal components.

5. Write a Python program to implement the Apriori algorithm for association rule mining. Use a transaction dataset to generate frequent item sets and association rules. Display support, confidence, and lift values, and analyse how minimum support and confidence thresholds affect the results.

Aim

To implement the Apriori algorithm for association rule mining and calculate support, confidence, and lift.

Theory

Apriori is an unsupervised learning algorithm used to discover relationships between items in transactional datasets.

Support indicates how frequently an itemset occurs.

Confidence indicates how often the consequent occurs when the antecedent occurs.

Lift indicates the strength of the relationship between two items.

Important Formulas

Support: $\text{Support}(X) = \text{Transactions containing } X / \text{Total transactions}$

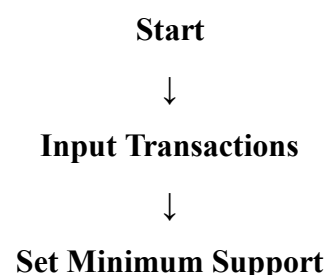
Confidence: $\text{Confidence}(X \rightarrow Y) = \text{Support}(X \cup Y) / \text{Support}(X)$

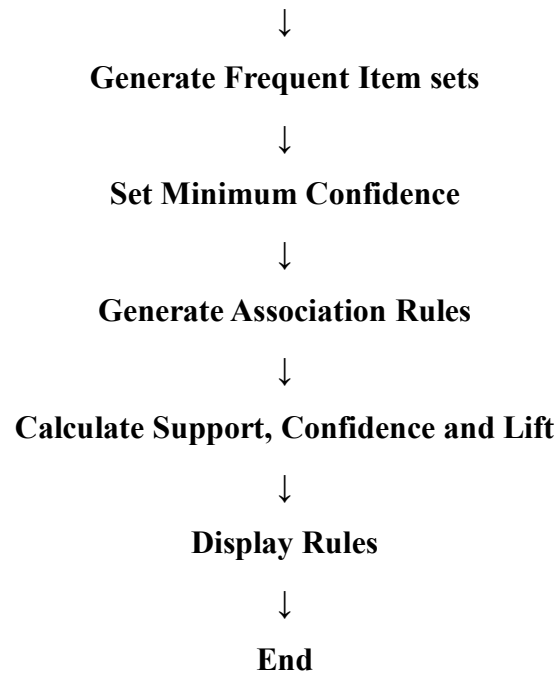
Lift: $\text{Lift}(X \rightarrow Y) = \text{Confidence}(X \rightarrow Y) / \text{Support}(Y)$

Algorithm

1. Create the transaction dataset.
2. Convert transactions into binary format.
3. Set minimum support.
4. Generate frequent item sets.
5. Set minimum confidence.
6. Generate association rules.
7. Calculate support, confidence, and lift.
8. Analyse the generated rules.

Apriori Algorithm





Python Program

```
[5] ✓ On data = te.fit(transactions).transform(transactions)
import pandas as pd
df = pd.DataFrame(data, columns=te.columns_)
frequent_itemsets = apriori(
    df,
    min_support=0.5,
    use_colnames=True
)
print("Frequent Itemsets:")
print(frequent_itemsets)
rules = association_rules(
    frequent_itemsets,
    metric="confidence",
    min_threshold=0.6
)
print("\nAssociation Rules:")
print(
    rules[
        ["antecedents", "consequents",
         "support", "confidence", "lift"]]
)
```

*** Frequent Itemsets:

support	itemsets
0 1.000000	(Bread)
1 0.666667	(Butter)
2 0.666667	(Milk)
3 0.666667	(Bread, Butter)
4 0.666667	(Bread, Milk)

Association Rules:

antecedents	consequents	support	confidence	lift
0 (Bread)	(Butter)	0.666667	0.666667	1.0
1 (Butter)	(Bread)	0.666667	1.000000	1.0
2 (Bread)	(Milk)	0.666667	0.666667	1.0
3 (Milk)	(Bread)	0.666667	1.000000	1.0

Analysis

- Increasing minimum support produces fewer frequent itemsets.
- Decreasing minimum support produces more itemsets.
- Increasing minimum confidence produces fewer but stronger rules.
- Lift greater than 1 indicates a positive association.
- Apriori is commonly used in market basket analysis and recommendation systems.

Result

The Apriori algorithm was successfully implemented to generate frequent itemsets and association rules using support, confidence, and lift.